

Case Study: Product Order Management System

1. Case Study Overview

The existing application currently manages products using Spring Boot, Spring Data JPA, and PostgreSQL.

The company now wants to add an **Order Management module** so customers can place orders for available products.

The new module must support:

- Creating an order
 - Viewing an order by ID
 - Viewing all orders
 - Updating an order
 - Cancelling or deleting an order
 - Checking product availability
 - Calculating the order amount
 - Updating product stock
-

2. Business Scenario

An electronics store maintains products such as laptops, mobile phones, keyboards, and monitors.

Each product has:

- Product ID
- Product name
- Price
- Available quantity
- Category

A customer can place an order for a particular product.

For example:

- Product: Dell Laptop
- Product price: ₹65,000
- Available quantity: 10
- Customer order quantity: 2
- Total order amount: ₹1,30,000

After the order is successfully created, the available product quantity must reduce from 10 to 8.

3. Order Information

The Order entity should contain the following information.

Field	Description
Order ID	Unique identifier generated by the database
Customer Name	Name of the customer placing the order
Customer Email	Email address of the customer
Customer Mobile	Customer contact number
Order Date	Date on which the order was placed
Product	Product selected by the customer
Ordered Quantity	Number of product units ordered
Product Price	Price of one product at the time of ordering
Total Amount	Product price multiplied by ordered quantity
Order Status	Current status of the order
Shipping Address	Address where the order must be delivered

4. Order Status Values

The application should support the following order statuses:

- CREATED
- CONFIRMED
- SHIPPED
- DELIVERED
- CANCELLED

When a new order is created, its initial status should automatically be set to `CREATED`.

5. Relationship Between Product and Order

Each order is associated with one product.

A single product may appear in multiple orders.

Example:

Order ID	Customer	Product
101	Amit	Dell Laptop
102	Sneha	Dell Laptop
103	Rahul	Wireless Mouse

This represents:

- One Product can have many Orders.
- Each Order belongs to one Product.

The Order entity should therefore maintain a reference to the Product entity.

6. Create Order Requirement

Objective

Allow a customer to place an order for an existing product.

Input Details

The user should provide:

- Customer name
- Customer email
- Customer mobile
- Product ID
- Required quantity
- Shipping address

The user should not directly provide:

- Order ID
- Product price
- Total amount
- Order date
- Order status

These values should be generated or calculated by the application.

Create Order Processing Steps

1. Receive the order request.
2. Validate the customer details.
3. Validate the product ID.
4. Retrieve the product from the Product Repository.
5. Verify that the product exists.
6. Check whether the ordered quantity is greater than zero.
7. Check whether sufficient product quantity is available.
8. Obtain the current product price.
9. Calculate the total order amount.
10. Set the current date as the order date.
11. Set the order status to `CREATED`.
12. Save the order in the database.
13. Reduce the product quantity.
14. Save the updated product.
15. Return the created order details.

Example

Product details before ordering:

- Product ID: 1
- Product name: Laptop
- Price: ₹65,000
- Available quantity: 10

Customer orders:

- Quantity: 2

Order calculation:

$\text{Total Amount} = \text{Product Price} \times \text{Ordered Quantity}$

$\text{Total Amount} = 65,000 \times 2$

$\text{Total Amount} = ₹1,30,000$

Updated product quantity:

$\text{Available Quantity} = 10 - 2$

$\text{Available Quantity} = 8$

7. Create Order Validations

The following validations must be applied.

Customer Name

- Must not be null.
- Must not be blank.
- Should contain at least three characters.

Customer Email

- Must not be null.
- Must be a valid email address.

Customer Mobile

- Must contain ten digits.
- Must not contain alphabetic characters.

Product ID

- Must not be null.
- Must be greater than zero.
- The product must exist in the database.

Ordered Quantity

- Must not be null.
- Must be greater than zero.

- Must not be greater than the available product quantity.

Shipping Address

- Must not be null.
 - Must not be blank.
 - Should contain sufficient delivery information.
-

8. Insufficient Stock Scenario

Suppose the product contains only five units.

The customer requests eight units.

The system must reject the order.

Expected error message:

```
Insufficient stock.
```

```
Requested quantity: 8
```

```
Available quantity: 5
```

The order must not be saved, and the product quantity must not change.

9. Product Not Found Scenario

Suppose the customer sends Product ID 200, but no such product exists.

The system must reject the order.

Expected error message:

```
Product not found with ID: 200
```

10. Get Order by ID Requirement

Objective

Retrieve complete details of a particular order.

Processing Steps

1. Receive the order ID.
2. Validate that the order ID is not null.
3. Validate that the order ID is greater than zero.
4. Search for the order using the Order Repository.
5. Return the order when found.
6. Throw an Order Not Found exception when the order does not exist.

Expected Details

The response should include:

- Order ID
- Customer details
- Product details
- Ordered quantity
- Product price
- Total amount
- Order date
- Order status
- Shipping address

Error Scenario

Order not found with ID: 101

11. Get All Orders Requirement

Objective

Retrieve all orders stored in the system.

Processing Steps

1. Request all orders from the Order Repository.
2. Return the list of orders.
3. If no orders exist, return an empty list.

Optional Enhancements

Later, the application may support:

- Pagination
 - Sorting by order date
 - Sorting by total amount
 - Filtering by status
 - Filtering by customer email
 - Filtering by product
 - Filtering by date range
-

12. Update Order Requirement

The update operation should not allow every order field to be modified.

The following fields may be updated:

- Customer mobile
- Shipping address
- Order status

The following fields should normally not be updated directly:

- Order ID
- Product
- Ordered quantity
- Product price
- Total amount

- Order date

Update Processing Steps

1. Receive the order ID and updated order details.
 2. Validate the order ID.
 3. Retrieve the existing order.
 4. Verify that the order exists.
 5. Check the current order status.
 6. Apply only permitted changes.
 7. Validate the new values.
 8. Save the updated order.
 9. Return the updated order.
-

13. Order Status Transition Rules

The application should not allow arbitrary status changes.

Valid status changes:

CREATED → CONFIRMED

CONFIRMED → SHIPPED

SHIPPED → DELIVERED

CREATED → CANCELLED

CONFIRMED → CANCELLED

Invalid status changes:

DELIVERED → CREATED

DELIVERED → CANCELLED

CANCELLED → SHIPPED

SHIPPED → CREATED

Example error message:

Order status cannot be changed from DELIVERED to CREATED.

14. Update Shipping Address Rules

The shipping address may be changed only when the order status is:

- CREATED
- CONFIRMED

The address must not be changed after the order has been shipped.

Invalid scenario:

Current order status: SHIPPED
Requested operation: Change shipping address

Expected error:

Shipping address cannot be changed after the order has been shipped.

15. Cancel Order Requirement

Instead of immediately deleting an order, the application should preferably mark it as **CANCELLED**.

Cancellation Processing Steps

1. Receive the order ID.
2. Retrieve the order.

3. Verify that the order exists.
4. Check the current order status.
5. Reject cancellation when the order is already shipped or delivered.
6. Change the order status to **CANCELLED**.
7. Restore the ordered quantity to the product stock.
8. Save the updated product.
9. Save the cancelled order.
10. Return the cancellation confirmation.

Example

Product quantity before cancellation:

Available quantity: 8

Cancelled order quantity:

Ordered quantity: 2

Product quantity after cancellation:

Updated quantity = $8 + 2$

Updated quantity = 10

16. Delete Order Requirement

Permanent deletion should normally be restricted.

An order may be permanently deleted only when:

- It is already cancelled.
- It was created as a test or incorrect record.

- The business permits permanent deletion.

Delete Processing Steps

1. Receive the order ID.
2. Validate the order ID.
3. Retrieve the order.
4. Verify that the order exists.
5. Check whether the order status is `CANCELLED`.
6. Reject deletion when the order is active.
7. Delete the cancelled order.

Invalid Delete Example

Order status: `SHIPPED`

Expected error:

Only cancelled orders can be deleted.

17. Order Repository Responsibilities

Create an `OrderRepository` interface using Spring Data JPA.

The repository should initially support:

- Save an order
- Find an order by ID
- Find all orders
- Check whether an order exists
- Delete an order

Additional repository methods may later support:

- Find orders by customer email

- Find orders by status
 - Find orders by product ID
 - Find orders between two dates
 - Find orders with total amount greater than a value
-

18. Order Service Interface Responsibilities

Create an `OrderService` interface.

The service contract should contain operations for:

1. Create an order.
2. Get an order by ID.
3. Get all orders.
4. Update permitted order details.
5. Update order status.
6. Cancel an order.
7. Delete a cancelled order.

The interface should only define operation contracts.

It should not contain business implementation logic.

19. Order Service Implementation Responsibilities

Create an `OrderServiceImpl` class.

The service implementation should handle:

- Customer data validation
- Product retrieval
- Product availability verification
- Price retrieval
- Total amount calculation
- Order date generation
- Default status assignment

- Status transition validation
- Product stock reduction
- Product stock restoration
- Order creation
- Order update
- Order cancellation
- Order deletion
- Exception handling

The service layer should contain the main business logic.

20. Order Controller Responsibilities

Create an `OrderController`.

The controller should expose REST endpoints for order operations.

Suggested API operations:

Operation	HTTP Method	Endpoint
Create order	POST	<code>/api/orders</code>
Get order by ID	GET	<code>/api/orders/{id}</code>
Get all orders	GET	<code>/api/orders</code>
Update order	PUT	<code>/api/orders/{id}</code>
Update status	PATCH	<code>/api/orders/{id}/status</code>
Cancel order	PATCH	<code>/api/orders/{id}/cancel</code>
Delete order	DELETE	<code>/api/orders/{id}</code>

The controller should:

- Receive HTTP requests.
 - Validate request data.
 - Call the Order Service.
 - Return appropriate HTTP responses.
 - Avoid containing business logic.
-

21. Suggested HTTP Status Codes

Scenario	HTTP Status
Order successfully created	201 Created
Order successfully retrieved	200 OK
Order successfully updated	200 OK
Order successfully cancelled	200 OK
Order successfully deleted	204 No Content
Invalid request data	400 Bad Request
Product not found	404 Not Found
Order not found	404 Not Found
Insufficient product quantity	409 Conflict
Invalid order status transition	409 Conflict

22. Required Exception Classes

The Order module may use the following custom exceptions:

OrderNotFoundException

Used when an order does not exist.

ProductNotFoundException

The existing Product exception can be reused.

InsufficientStockException

Used when the requested quantity exceeds available quantity.

InvalidOrderStatusException

Used when an invalid status transition is attempted.

OrderCancellationException

Used when a shipped or delivered order is cancelled.

OrderDeletionException

Used when an active order is permanently deleted.

23. Database Tables

The existing application already contains the `products` table.

A new `orders` table should be added.

Products Table

Important columns:

- id
- name
- description
- price
- quantity
- category

Orders Table

Important columns:

- id
- customer_name
- customer_email
- customer_mobile
- order_date
- ordered_quantity
- product_price
- total_amount
- order_status
- shipping_address
- product_id

The `product_id` column should act as a foreign key referencing the Product table.

24. Transaction Requirement

Order creation involves two database changes:

1. Saving the order.
2. Updating the product quantity.

Both operations should execute as one transaction.

If saving the order fails, the product quantity must not be reduced.

If updating the product fails, the order must not be saved.

The same rule applies during cancellation:

1. Update the order status.
2. Restore the product quantity.

Both changes must succeed together.

25. Detailed Case Study Test Data

Existing Products

ID	Product	Price	Quantity
1	Dell Laptop	₹65,000	10
2	Wireless Mouse	₹1,200	25
3	Mechanical Keyboard	₹4,500	15
4	LED Monitor	₹18,000	8

26. Test Scenario 1: Successful Order

Customer:

- Name: Amit Sharma
- Email: amit@example.com
- Mobile: 9876543210
- Product ID: 1
- Quantity: 2

- Address: Wakad, Pune

Expected result:

- Order should be created.
 - Product price should be ₹65,000.
 - Total amount should be ₹1,30,000.
 - Order status should be CREATED.
 - Product quantity should reduce from 10 to 8.
-

27. Test Scenario 2: Product Does Not Exist

Input:

- Product ID: 100
- Quantity: 1

Expected result:

- Order creation should fail.
 - Product Not Found exception should occur.
 - No order should be saved.
-

28. Test Scenario 3: Insufficient Stock

Existing product quantity:

8

Requested quantity:

10

Expected result:

- Order creation should fail.
 - Insufficient Stock exception should occur.
 - Product quantity should remain 8.
-

29. Test Scenario 4: Invalid Quantity

Input:

Ordered quantity: 0

Expected result:

- Validation should fail.
- Order should not be created.

The same validation should fail for negative quantities.

30. Test Scenario 5: Cancel Created Order

Order status:

CREATED

Ordered quantity:

2

Expected result:

- Status should change to CANCELLED.
 - Ordered quantity should be restored to product stock.
-

31. Test Scenario 6: Cancel Delivered Order

Order status:

DELIVERED

Expected result:

- Cancellation should fail.

- Product quantity should not change.
 - Order status should remain DELIVERED.
-

32. Test Scenario 7: Update Order Status

Current status:

CREATED

New status:

CONFIRMED

Expected result:

- Status update should succeed.
-

33. Test Scenario 8: Invalid Status Update

Current status:

DELIVERED

Requested status:

SHIPPED

Expected result:

- Status update should fail.
 - Invalid Order Status exception should occur.
-

34. Implementation Sequence

Follow this order while adding the module.

Step 1: Create the Order Status Enum

Define all supported order statuses.

Step 2: Create the Order Entity

Add order fields, validations, and the Product relationship.

Step 3: Create Order Request Data Structure

Decide which values the customer should submit.

The request should mainly contain:

- Customer details
- Product ID
- Quantity
- Shipping address

Step 4: Create the Order Repository

Extend the appropriate Spring Data JPA repository interface.

Step 5: Create Custom Exceptions

Create exceptions for order-not-found, stock, cancellation, and status errors.

Step 6: Create the Order Service Interface

Define all required order operations.

Step 7: Create the Order Service Implementation

Implement validations, calculations, stock updates, and order processing.

Step 8: Add Transaction Management

Ensure order and product changes execute together.

Step 9: Create the Order Controller

Expose REST endpoints for all Order CRUD operations.

Step 10: Add Centralized Exception Handling

Return consistent error messages and HTTP status codes.

Step 11: Run Database Migration

Allow JPA to create the Order table or create it through a database migration tool.

Step 12: Test Create Order

Verify amount calculation and product quantity reduction.

Step 13: Test Read Operations

Test getting one order and all orders.

Step 14: Test Update Operations

Test allowed and prohibited modifications.

Step 15: Test Cancellation

Verify that product stock is restored correctly.

Step 16: Test Delete Operation

Verify that only eligible orders can be deleted.

35. Optional Advanced Requirements

After completing the basic Order CRUD, add the following enhancements:

- Search orders by customer email
- Search orders by status
- Search orders by product
- Search orders by order date
- Sort orders by total amount
- Pagination for order listing
- Order summary report
- Daily sales report
- Total revenue calculation
- Most ordered product

- Customer order history
 - Prevent duplicate order submission
 - Add created and updated timestamps
 - Add payment status
 - Add delivery tracking number
 - Add order audit history
-

36. Expected Learning Outcomes

After completing this case study, learners should understand:

- How to add a new module to an existing Spring Boot project
- How to create relationships between JPA entities
- How Controller, Service, Repository, and Entity layers interact
- How to validate business requirements
- How to calculate derived values
- How to update related entity data
- How transaction management works
- How to design proper REST endpoints
- How to handle business exceptions
- How Product and Order CRUD operations work together